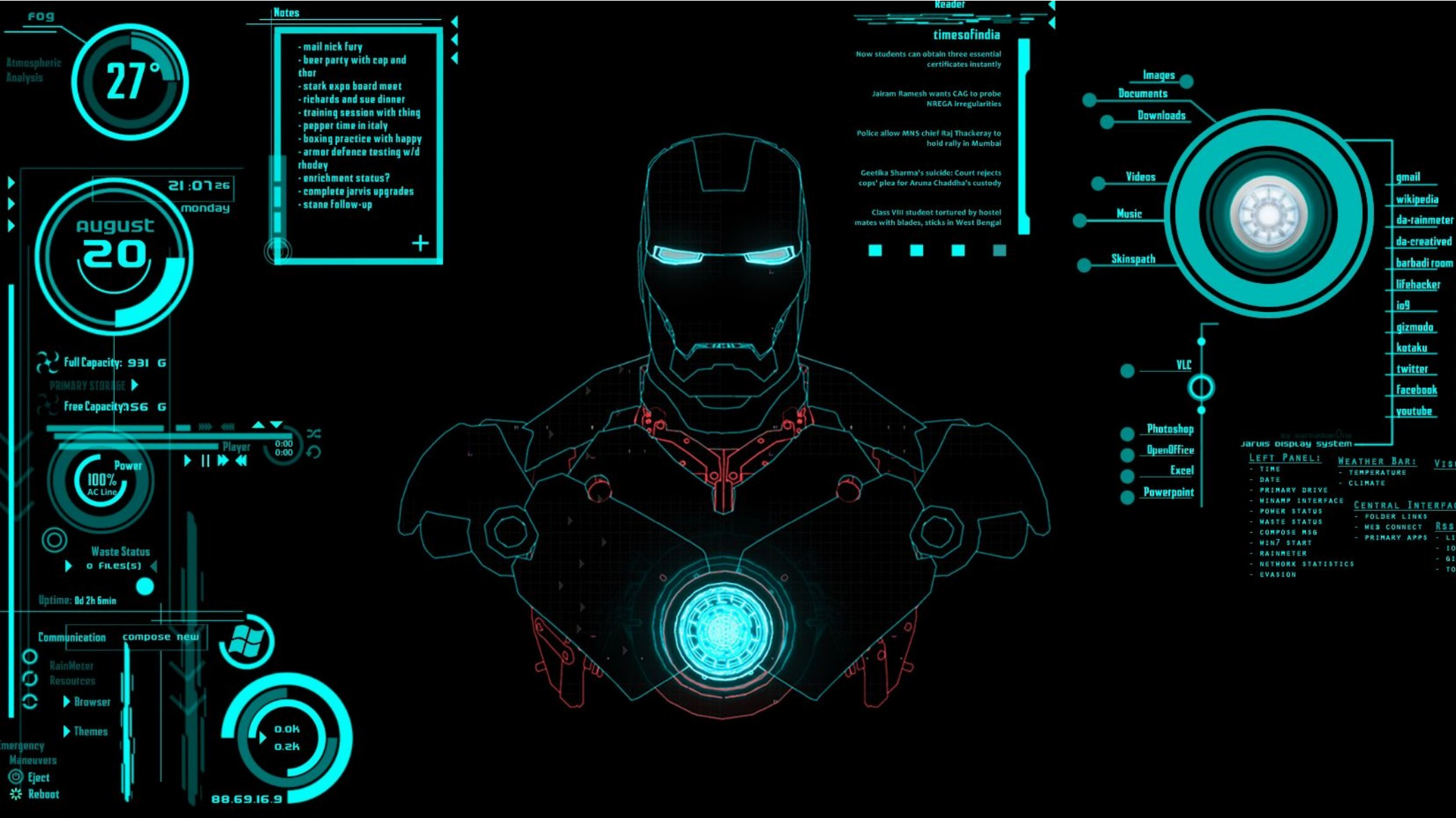




03.Desenhando em 2D – Parte 2

Prof. Alexandre Krohn

Roteiro

- Objetos paramétricos
 - Círculos
 - Curvas de Bézier
- Preenchimento com cores
 - Scan-fill
 - Flood-fill

Roteiro

- Objetos paramétricos
 - Círculos
 - Curvas de Bézier
- Preenchimento com cores
 - Scan-fill
 - Flood-fill

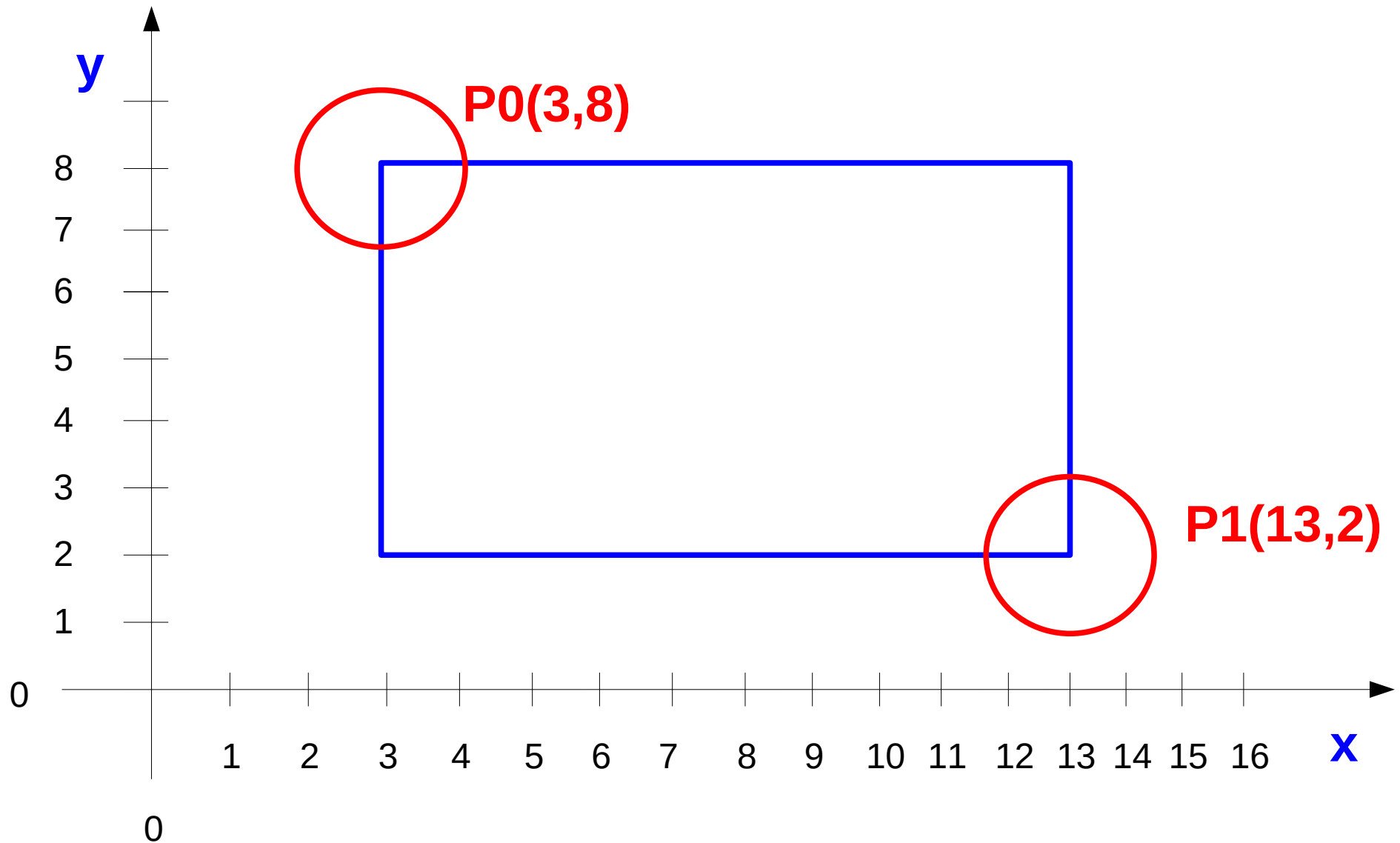
Objetos paramétricos

- Objetos paramétricos são aqueles para os quais **não há a necessidade do armazenamento** de todos os pontos que definem sua forma

Retângulo paramétrico

- Para desenharmos um retângulo, que é uma figura geométrica regular, só precisamos de duas coordenadas (**P0** e **P1**), e uma cor, sendo:
 - **P0** = canto superior esquerdo
 - **P1** = canto inferior direito

Retângulo paramétrico



Objetos paramétricos

- A definição de objetos paramétricos reduz a quantidade de memória necessária para sua representação

Roteiro

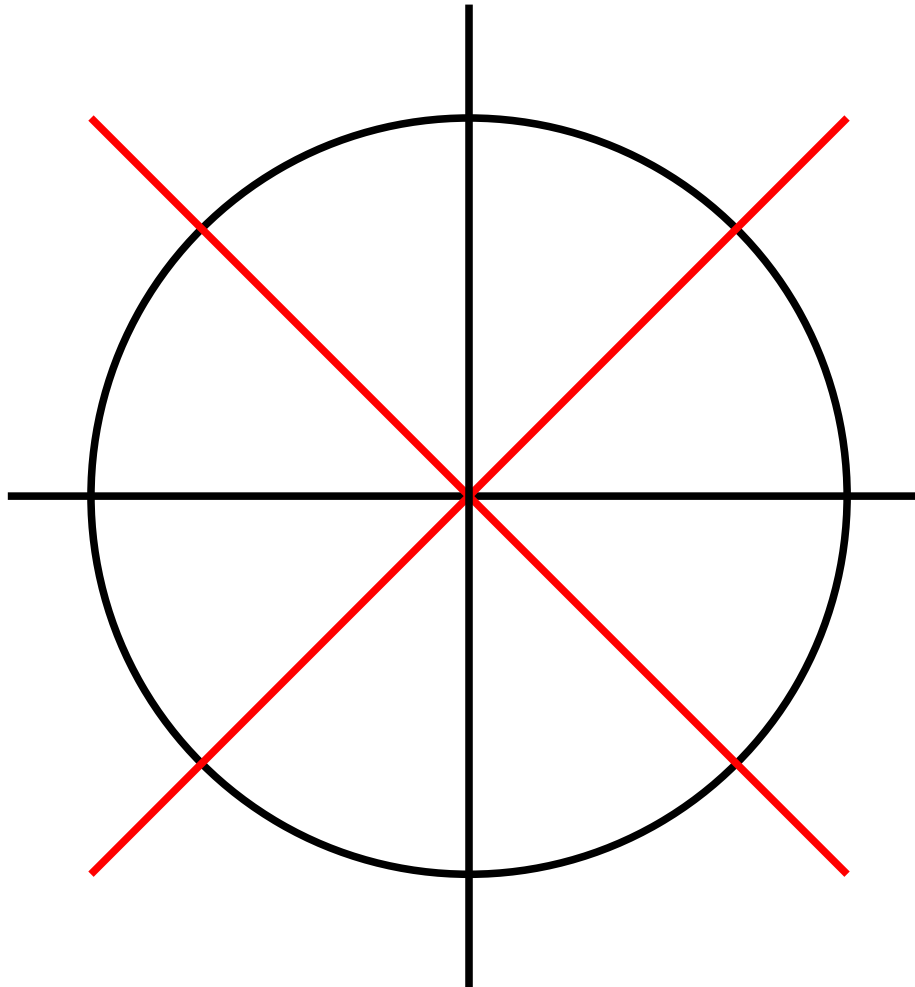
- Objetos paramétricos
 - Círculos
 - Curvas de Bézier
- Preenchimento com cores
 - Scan-fill
 - Flood-fill

Círculos de Bresenham

- **Bresenham** também elaborou um **algoritmo para o desenho de círculos**, seguindo um princípio semelhante ao do desenho de linhas retas
- Como dependem apenas das coordenadas do centro e de seu raio, também são objetos paramétricos.
- Assim como o algoritmo de retas, o algoritmo de círculos consegue realizar o desenho **evitando o uso de operações de ponto flutuante**.

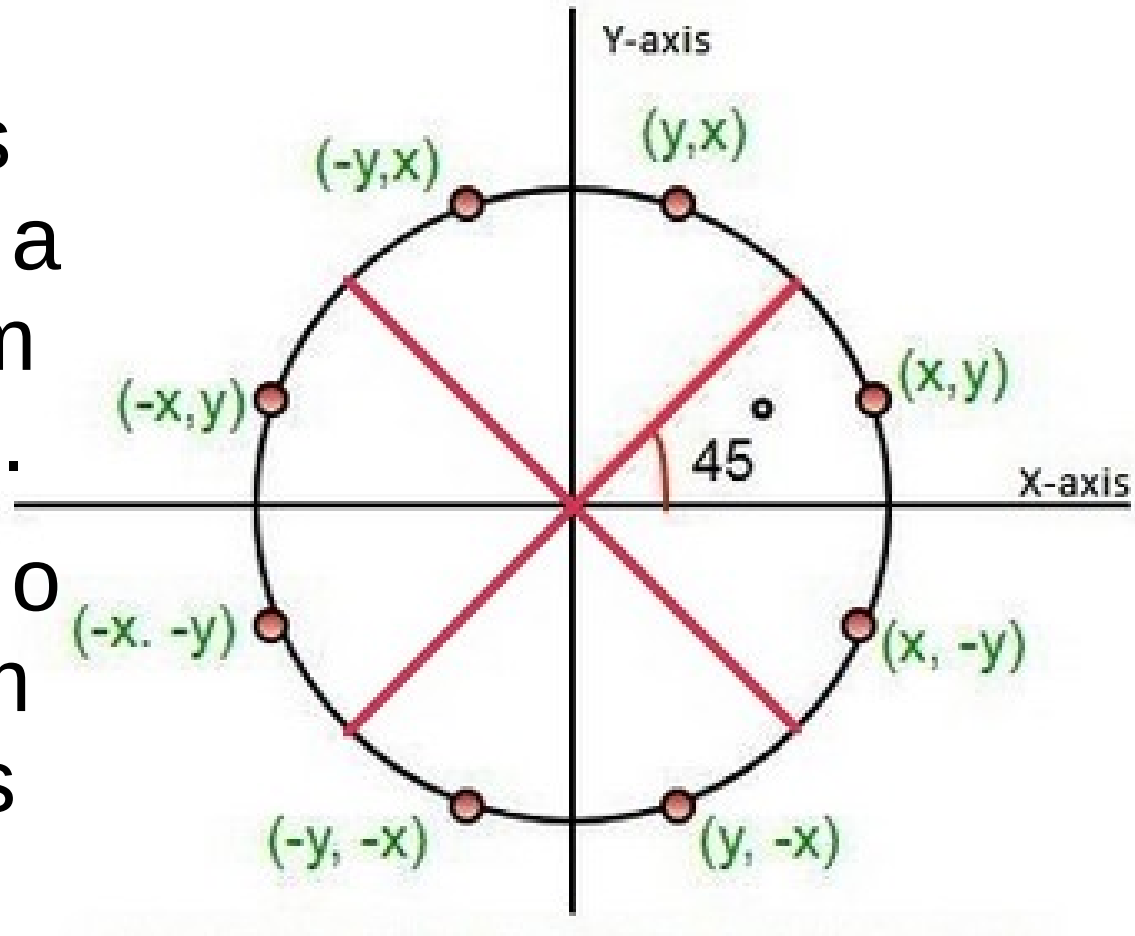
Círculos de Bresenham

- Baseando-se na propriedade de simetria dos círculos, o círculo pode ser dividido em 8 octantes de 45 graus



Círculos de Bresenham

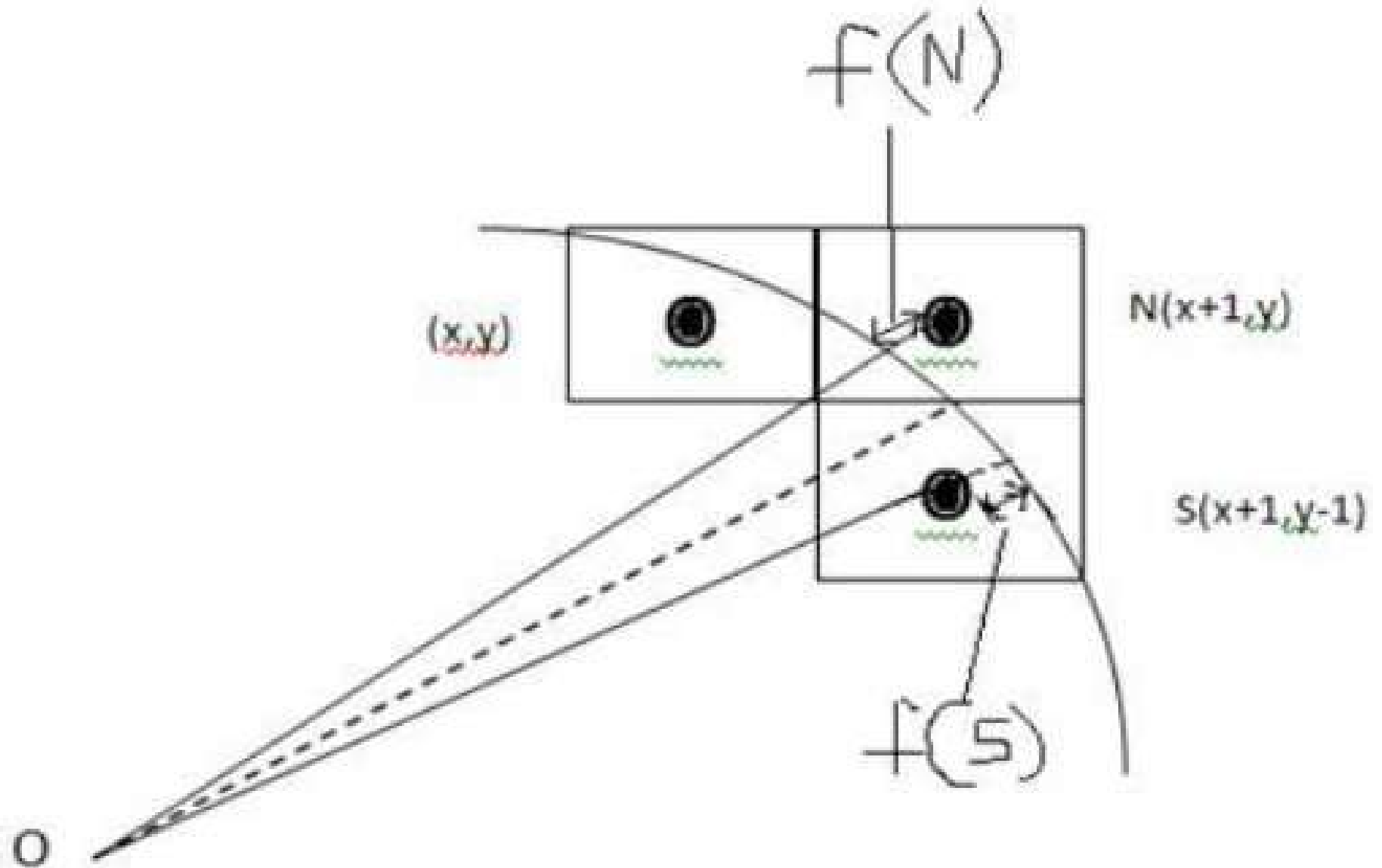
- O algoritmo calcula a localização dos pixels no primeiro octante e a estende para cada um dos outros 7 octantes.
- Para cada pixel (x,y) , o algoritmo desenha um pixel em cada um dos 8 octantes do círculo



Círculos de Bresenham

- Assumindo-se que o **centro da figura** é sua **origem**,
- O ponto (x,y) está no primeiro octante do círculo
- Calcula-se o próximo ponto usando:
 - **$n(x+1, y)$ ou $s(x+1, y-1)$**
- Determina-se um parâmetro de decisão **d** , com base no **raio** do círculo
 - Se **$d < 0$** , então **$n(x+1,y)$** é o próximo pixel,
 - Caso contrário **$s(x+1,y-1)$** é o próximo pixel

Círculos de Bresenham



Círculos de Bresenham : Código (1/2)

- Em código-fonte, temos:

```
void drawBresenhamCircle(int xc, int yc, int radius) {
    int x = 0, y = radius;
    int decisionParameter = 3 - 2 * radius;
    displayBresenhamCircle(xc, yc, x, y);
    while (y >= x)
    {
        x++;
        if (decisionParameter > 0)
        {
            y--;
            decisionParameter = decisionParameter + 4 * (x - y) + 10;
        }
        else
            decisionParameter = decisionParameter + 4 * x + 6;
        // mostra todos os 8 pixels de (x,y)
        displayBresenhamCircle(xc, yc, x, y);
    }
}
```

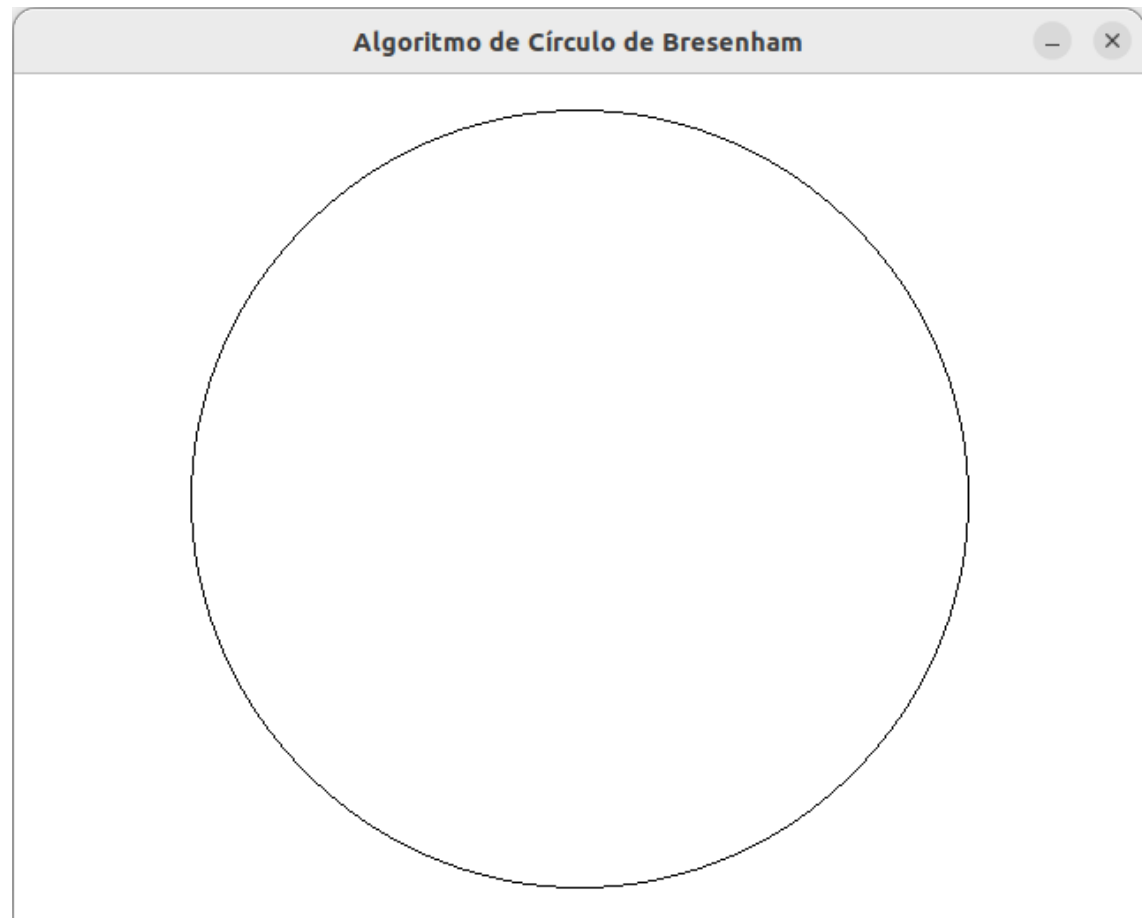
Círculos de Bresenham : Código (2/2)

- e:

```
void displayBresenhamCircle(int xc,int yc, int x, int y) {  
    // desenha todas as 8 coordenadas de(x,y),  
    // uma para cada octante  
    drawPixel(xc+x, yc+y, 0);  
    drawPixel(xc-x, yc+y, 0);  
    drawPixel(xc+x, yc-y, 0);  
    drawPixel(xc-x, yc-y, 0);  
    drawPixel(xc+y, yc+x, 0);  
    drawPixel(xc-y, yc+x, 0);  
    drawPixel(xc+y, yc-x, 0);  
    drawPixel(xc-y, yc-x, 0);  
}
```

Círculos de Bresenham : Exemplo

```
void display()  
{  
    drawBresenhamCircle(320,240,220);  
}
```



Roteiro

- Objetos paramétricos
 - Círculos
 - **Curvas de Bézier**
- Preenchimento com cores
 - Scan-fill
 - Flood-fill

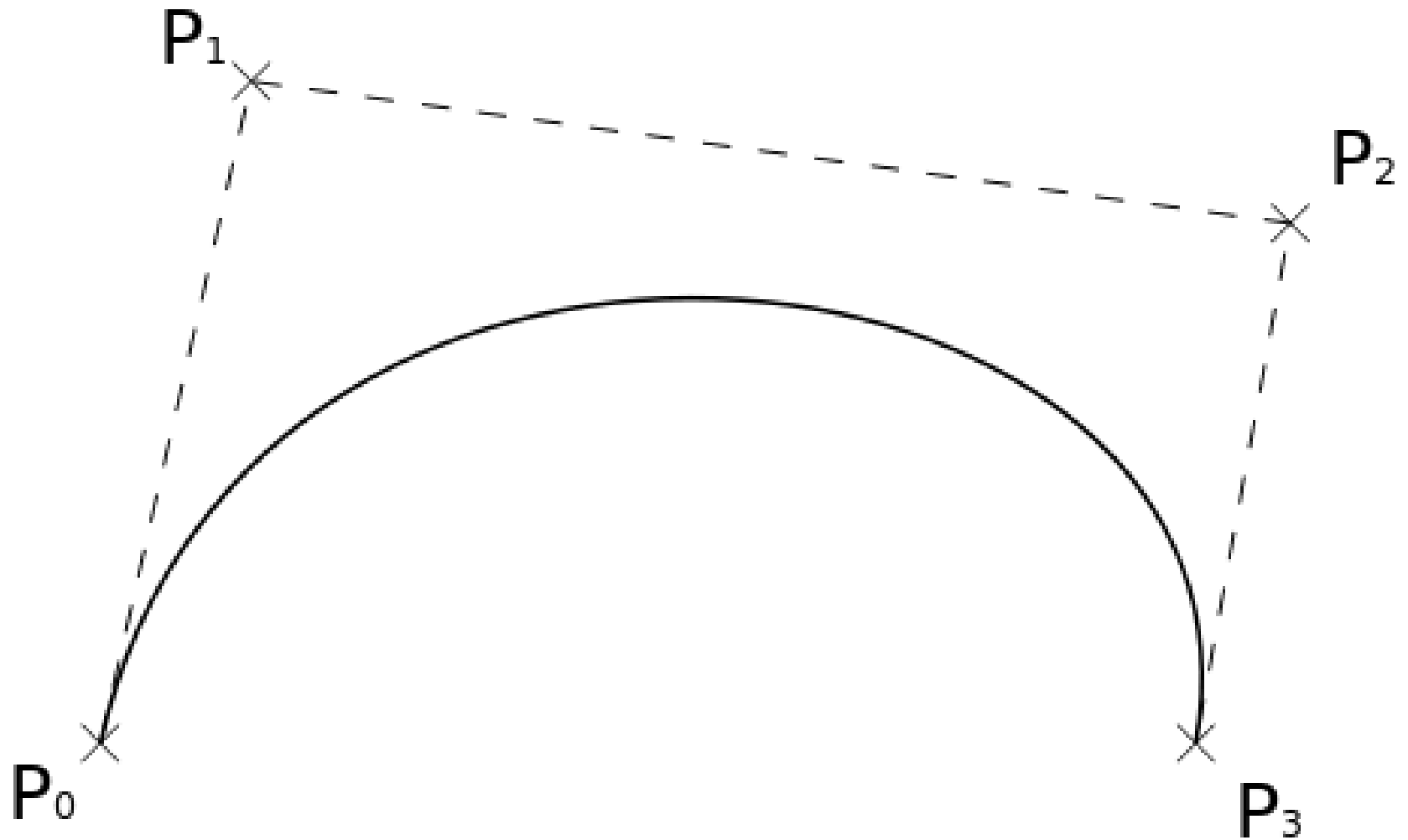
Curvas de Bézier

- Uma curva de Bézier (***Pierre Bézier***) é uma **curva definida matematicamente**, usada em aplicações gráficas bidimensionais como Adobe Illustrator, Inkscape, etc...
- A curva é definida por **quatro** pontos:
 - As “**âncoras**” : Os pontos **P0** e **P3**, que são as posições de início e de fim da curva,
 - As “**alças**”, **P1** e **P2**, que são dois pontos intermediários separados



Pierre Étienne Bézier (Paris, 1910 — 1999)

Curvas de Bézier



Matematicamente ?

- As **curvas de Bézier** podem ser geradas sob o controle de outros pontos.
- **Tangentes aproximadas** usando pontos de controle são usadas para gerar curvas.

Representação Matemática

- A curva de Bézier pode ser representada matematicamente como:

$$P(u) = \sum_{i=0}^n P_i B_i^n(u)$$

- Onde p_i representa o conjunto de pontos
- $B_i^n(u)$ representa os **polinômios de Bernstein** da função de combinação, dada por:

$$B_i^n(u) = \binom{n}{i} (1 - u)^{n-i} u^i$$

Representação Matemática

- Onde n é a ordem do polinômio
- i é o índice
- e u é a variável que pode conter entre 0 e 1

Polinômios de Bernstein?

- Em matemática, o teorema da aproximação de **Stone-Weierstrass** afirma que toda função real contínua cujo domínio é um intervalo compacto, ou seja, fechado e limitado pode ser aproximado uniformemente por polinômios.
- O polinômio de Bernstein (**Sergei Natanovich Bernstein**) é um desses polinômios

$$B_i^n(x) = \binom{n}{i} x^i (1 - x)^{n-i}$$



Sergei Natanovich Bernstein (Rússia, 1880 — 1968)

Polinômios de Bernstein

$$P_n(x) = \sum_{i=0}^n f\left(\frac{i}{n}\right) B_i^n(x)$$

Teorema de Stone-Weierstrass

Como $\sum_{i=0}^n B_i^n(x) = 1$, vale que $f(x) = \sum_{i=0}^n f(x) B_i^n(x)$ e vale a estimativa:

$$\begin{aligned} |f(x) - P_n(x)| &\leq \sum_{i=0}^n \left| f(x) - f\left(\frac{i}{n}\right) \right| B_i^n(x) \\ &= \sum_{S_1} \left| f(x) - f\left(\frac{i}{n}\right) \right| B_i^n(x) + \sum_{S_2} \left| f(x) - f\left(\frac{i}{n}\right) \right| B_i^n(x) \end{aligned}$$

onde $S_1 = \{0 \leq i \leq n : |x - i/n| < \delta\}$ e $S_2 = \{0 \leq i \leq n : |x - i/n| \geq \delta\}$.

$$\sum_{S_1} \left| f(x) - f\left(\frac{i}{n}\right) \right| B_i^n(x) \leq \sum_{S_1} \epsilon/2 B_i^n(x) \leq \epsilon/2 \sum_{i=1}^n B_i^n(x) = \epsilon/2$$

$$\begin{aligned} \sum_{S_2} \left| f(x) - f\left(\frac{i}{n}\right) \right| B_i^n(x) &\leq 2M \sum_{S_2} B_i^n(x) \leq 2M \sum_{S_2} \frac{(x - i/n)^2}{\delta^2} B_i^n(x) \\ &\leq \frac{2M}{\delta^2} \sum_{i=0}^n (x - i/n)^2 B_i^n(x) \leq \frac{M}{2\delta^2 n} < \epsilon/2, \text{ se } n > M/\epsilon\delta^2 \end{aligned}$$

E o resultado segue, escolhendo $n > M/\epsilon\delta^2$ e $P(x) := P_n(x)$.

Representação Matemática

- Uma curva de Bézier é definida por um conjunto de pontos de controle $P(0)$ até $P(n)$, onde n é chamado de ordem : ($n = 1$ para linear , $n = 2$ para quadrática, etc...).
- O primeiro e o último ponto de controle estão sempre nas extremidades da curvas, e os pontos intermediários geralmente não tocam a curva.
- Para uma curva de Bézier cúbica a ordem(n) do polinômio é 3, o índice i varia de $i = 0$ até $i = n$ e u é um numero decimal entre 0 e 1.

Representação Matemática

- Tendo a curva cúbica de Bézier definida como:

$$P(u) = P_0 B_0^3(u) + P_1 B_1^3(u) + P_2 B_2^3(u) + P_3 B_3^3(u)$$

- E a função de combinação da curva cúbica de Bézier definida como:

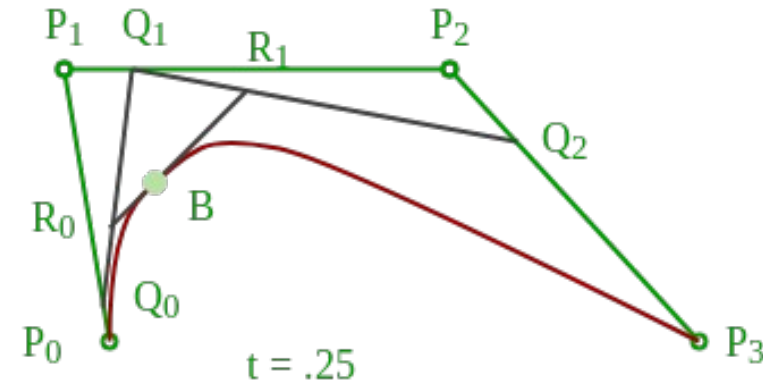
$$B_0^3(u) = \binom{3}{0} (1-u)^{3-0} u^0 \equiv 1(1-u)^3 u^0$$

$$B_1^3(u) = \binom{3}{1} (1-u)^{3-1} u^1 \equiv 3(1-u)^2 u^1$$

$$B_2^3(u) = \binom{3}{2} (1-u)^{3-2} u^2 \equiv 3(1-u)^1 u^2$$

$$B_3^3(u) = \binom{3}{3} (1-u)^{3-3} u^3 \equiv 1(1-u)^0 u^3$$

Representação Matemática



- Então

$$P(u) = (1 - u)^3 P_0 + 3u^1(1 - u)^2 P_1 + 3(1 - u)^1 u^2 P_2 + u^3 P_3$$

- e $P(u) = \{x(u), y(u)\}$

- Portanto

$$\begin{aligned} x(u) &= (1 - u)^3 x_0 + 3u^1(1 - u)^2 x_1 + 3(1 - u)^1 u^2 x_2 + u^3 x_3 \\ y(u) &= (1 - u)^3 y_0 + 3u^1(1 - u)^2 y_1 + 3(1 - u)^1 u^2 y_2 + u^3 y_3 \end{aligned}$$

- Calculamos a curva fazendo u variar em passos de 0,0001

Representação algorítmica

- Trazendo para algoritmos, e tendo os 4 pontos determinados por $p_0(x_0, y_0)$, $p_1(x_1, y_1)$, $p_2(x_2, y_2)$, $p_3(x_3, y_3)$, temos que:

```
para u de 0 ate 1 passo 0.0001 faca
```

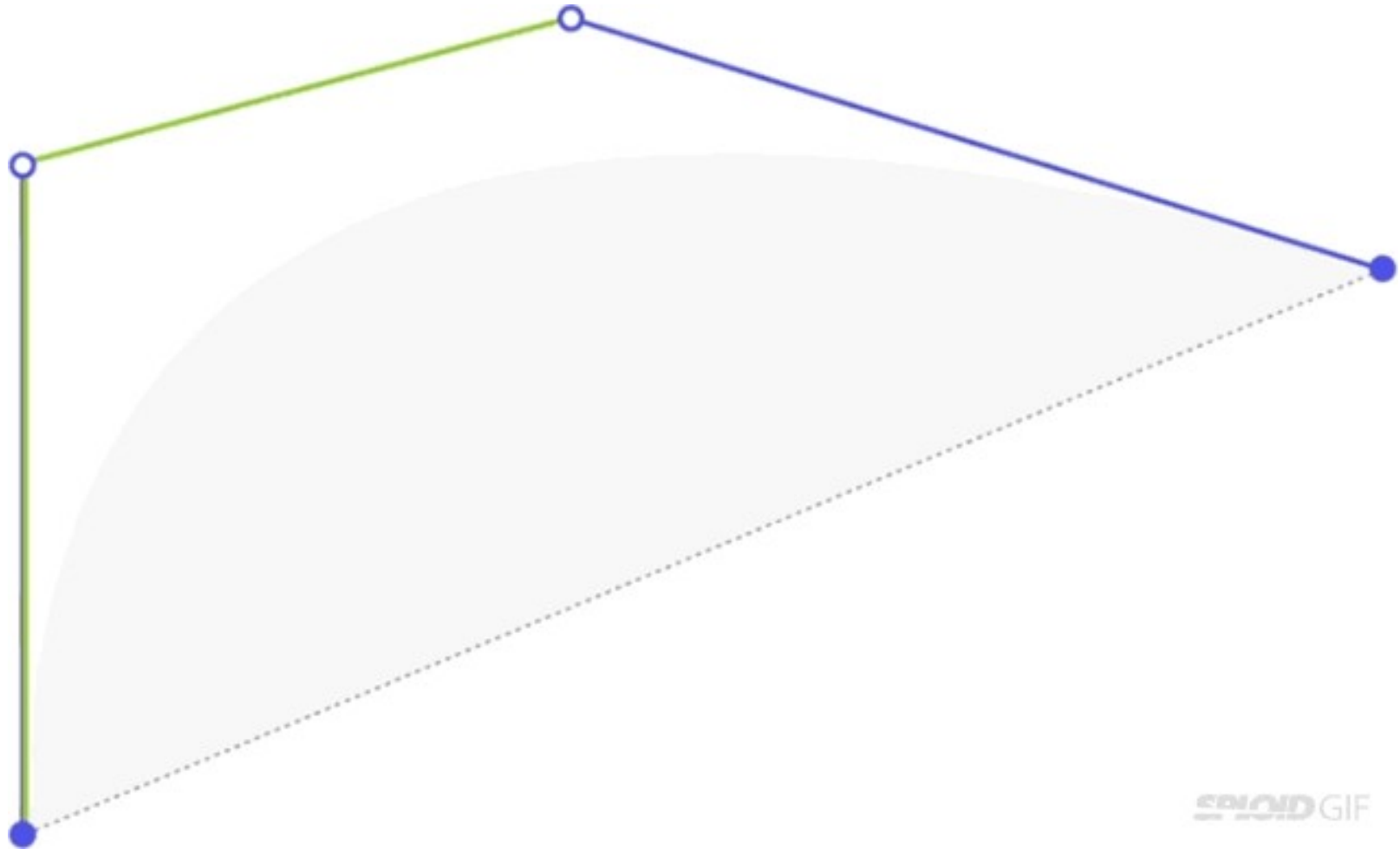
```
  xu = (1-u)^3*x0+3*u*(1-u)^2*x1+3*u^2*(1-u)*x2+u^3*x3
```

```
  yu = (1-u)^3*y0+3*u*(1-u)^2*y1+3*u^2*(1-u)*y2+u^3*y3
```

```
  desenha_pixel(xu, yu, cor)
```

```
fimpara
```

Representação Gráfica



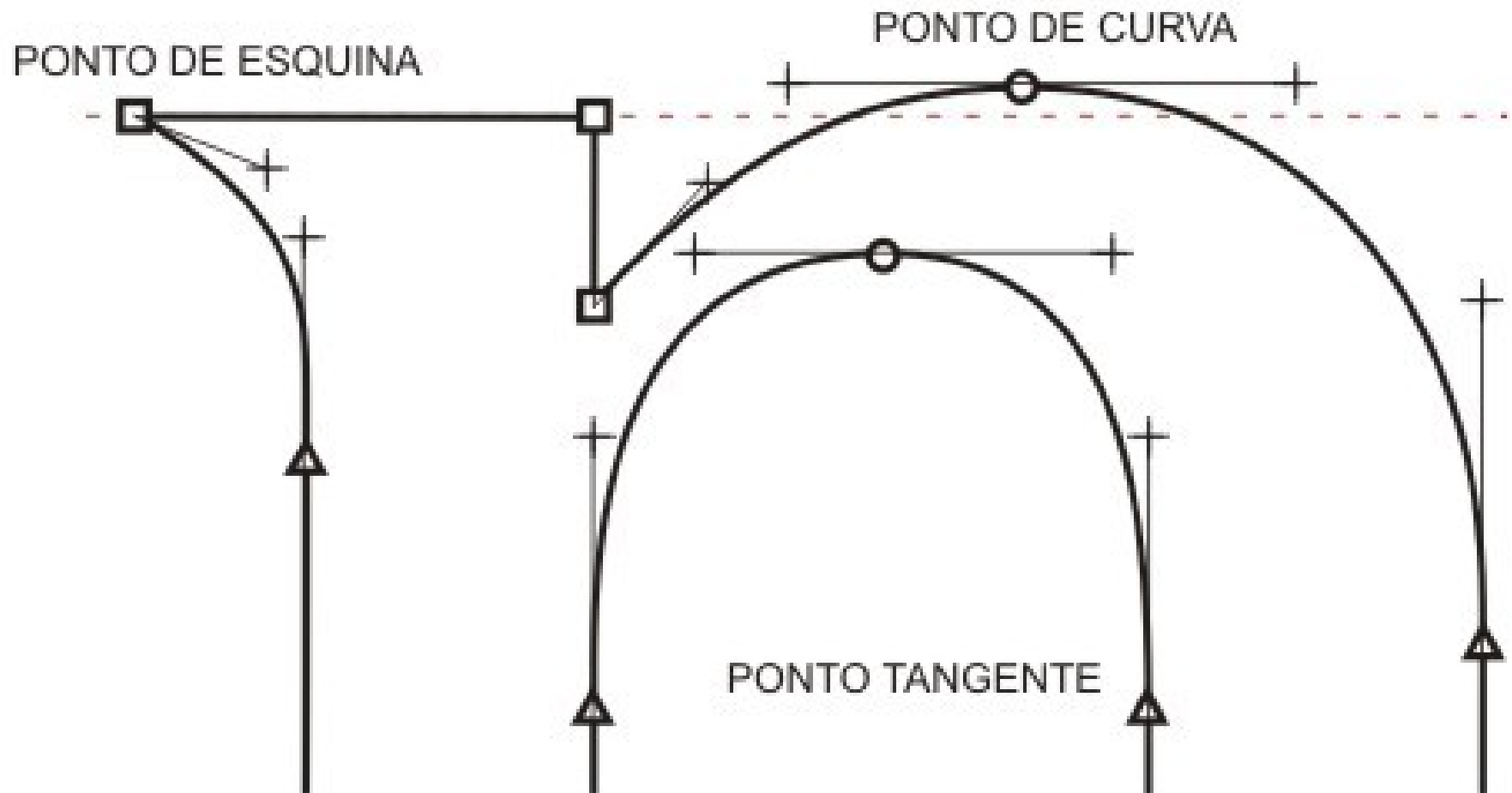
Para uma demonstração em vídeo, acesse : <https://vimeo.com/106757336>

Propriedades das curvas de Bézier

- **1)** Elas sempre passam pelos pontos inicial e final
- **2)** Estão sempre contidas na “casca” convexa formada por seus pontos de controle
- **3)** O grau do polinômio que define o segmento da curva é um a menos que o número de pontos de definição. Portanto, para 4 pontos de controle, o grau do polinômio é 3, portanto um polinômio cúbico.
- **4)** Uma curva de Bézier geralmente segue a forma do polígono formado pelos seus pontos de controle.
- **5)** A direção do vetor tangente ao ponto final é a mesma que o vetor determinado pelo primeiro e último segmentos.
- **6)** As curvas de Bézier possuem controle global, o que significa que mover um ponto de controle altera a forma de toda a curva

Curvas de Bézier : Usos:

- Tipografia:



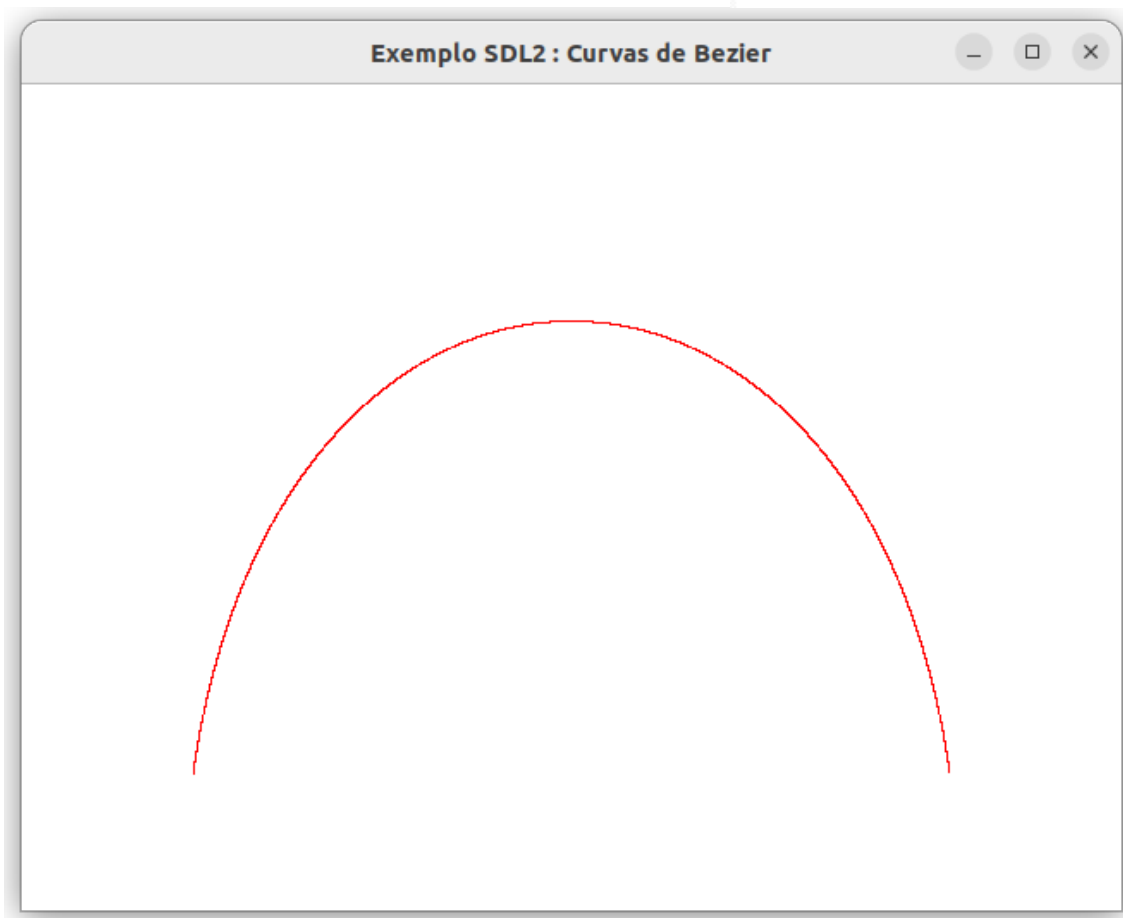
Curvas de Bézier :

Exemplo

```
void display()
{
    Uint32 vermelho = RGB(255,0,0);

    bool desenha_pontos = false;

    int x1[] = {100,150,490,540};
    int y1[] = {400,50,50,400};
    bezierCurve(x1,y1,desenha_pontos, vermelho);
}
```



Objetos Paramétricos:

Resumo

- Resumindo, para representar alguns objetos paramétricos, são necessários:
 - **Retângulo:** o ponto (x,y) superior esquerdo e o inferior direito, mais a cor, ou:
 - O ponto inferior esquerdo, mais a medida da altura, da largura e a cor
 - **Círculo:** O ponto central, o raio e a cor
 - **Curva de Bézier:** Os quatro pontos que definem a curva, mais a cor

Roteiro

- Objetos paramétricos
 - Círculos
 - Curvas de Bézier
- Preenchimento com cores
 - Scan-fill
 - Flood-fill

Preenchimento com cores

- Preencher um objeto (polígono) com cores é “pintá-lo” de uma determinada cor
- Esta operação deixa de ser simples quando os objetos tem formas irregulares
- A questão passa a ser: **Como implementar um algoritmo que seja genérico, isto é, preencha qualquer forma?**
- Assim como anti-aliasing, também há mais do que um algoritmo possível

Algoritmos de preenchimento

- Os algoritmos de preenchimento de polígonos variam em função de sua performance e do “conhecimento” do objeto que devem preencher
- Dois dos mais comuns são o algoritmo de *Scan-fill* e *Flood-Fill*
- Esses não são os únicos

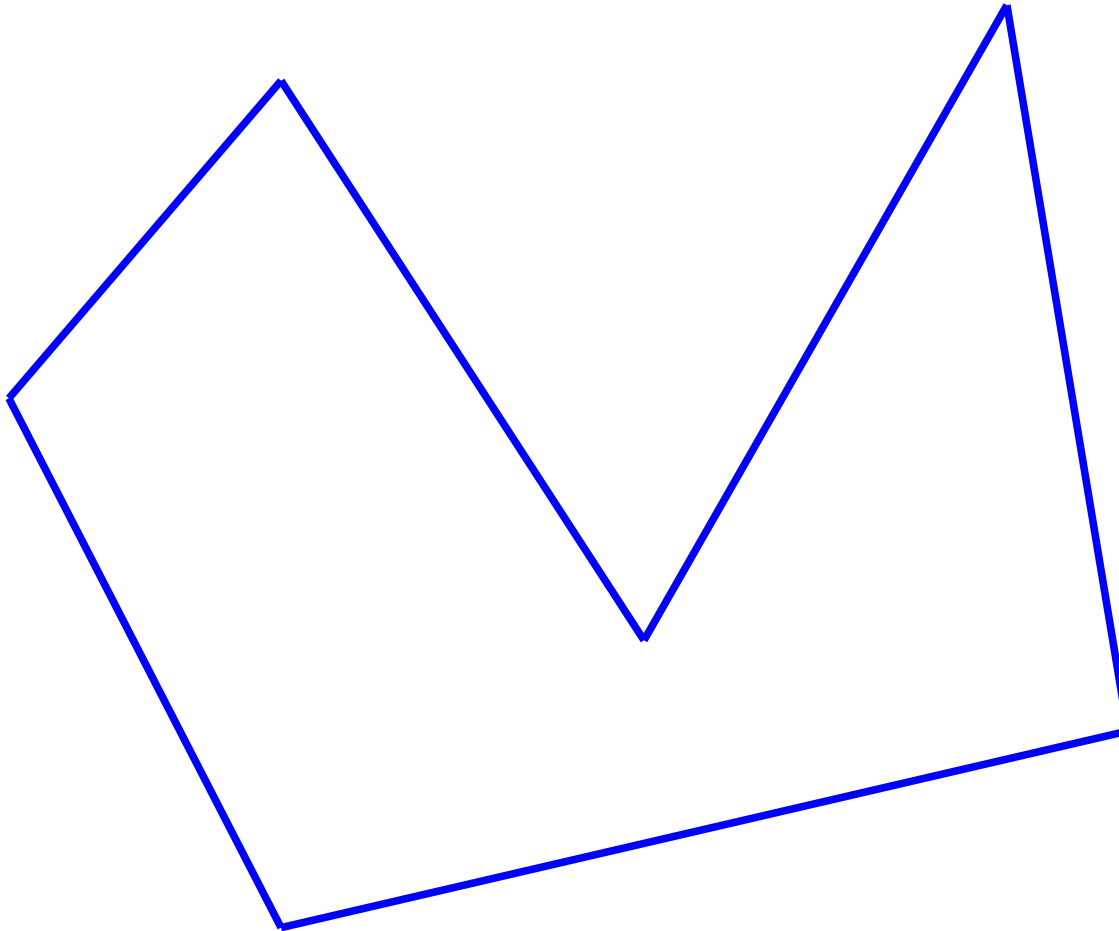
Roteiro

- Objetos paramétricos
 - Círculos
 - Curvas de Bézier
- Preenchimento com cores
 - Scan-fill
 - Flood-fill

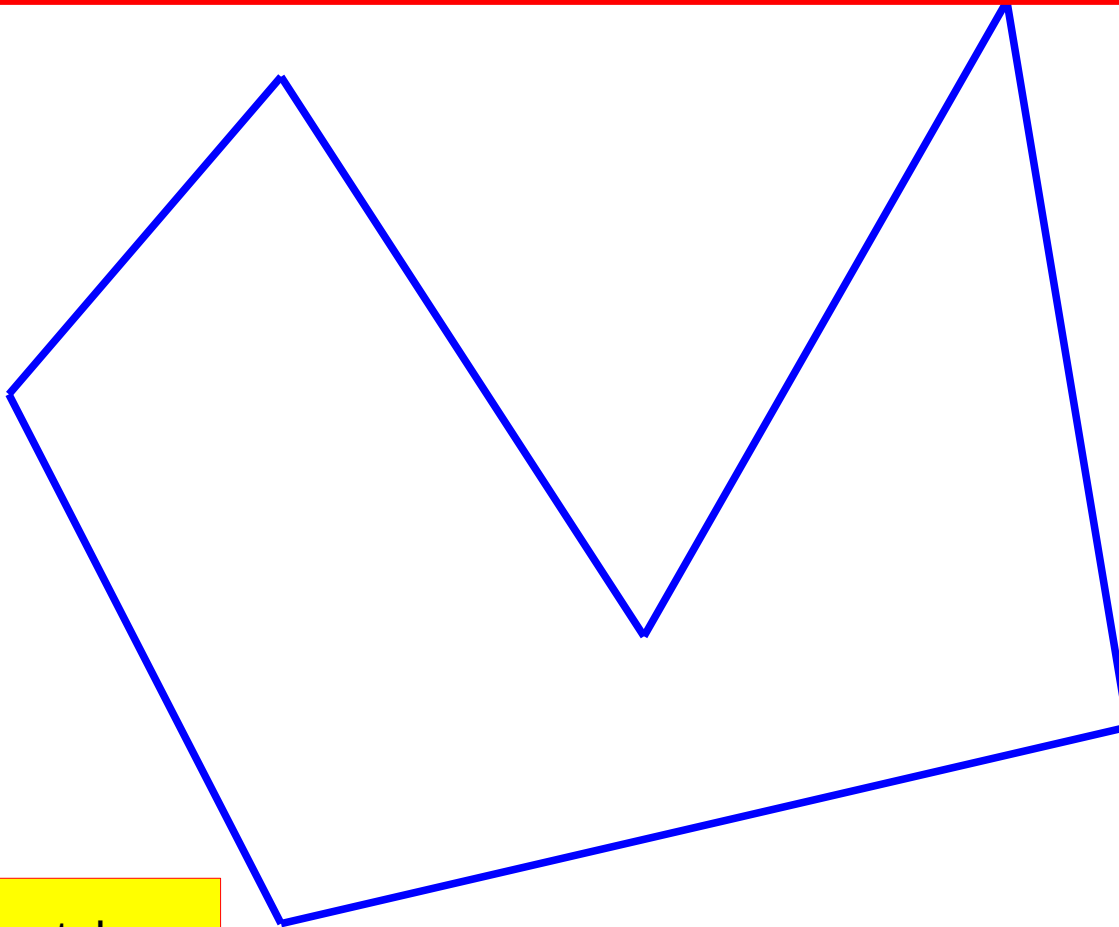
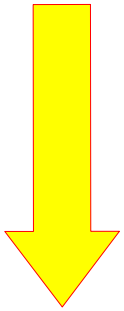
Algoritmo Scan-fill

- Provavelmente o **mais eficiente** algoritmo de preenchimento
- **Requer conhecimento prévio das informações (vértices e arestas) sobre o polígono a ser preenchido**

Algoritmo Scan-fill

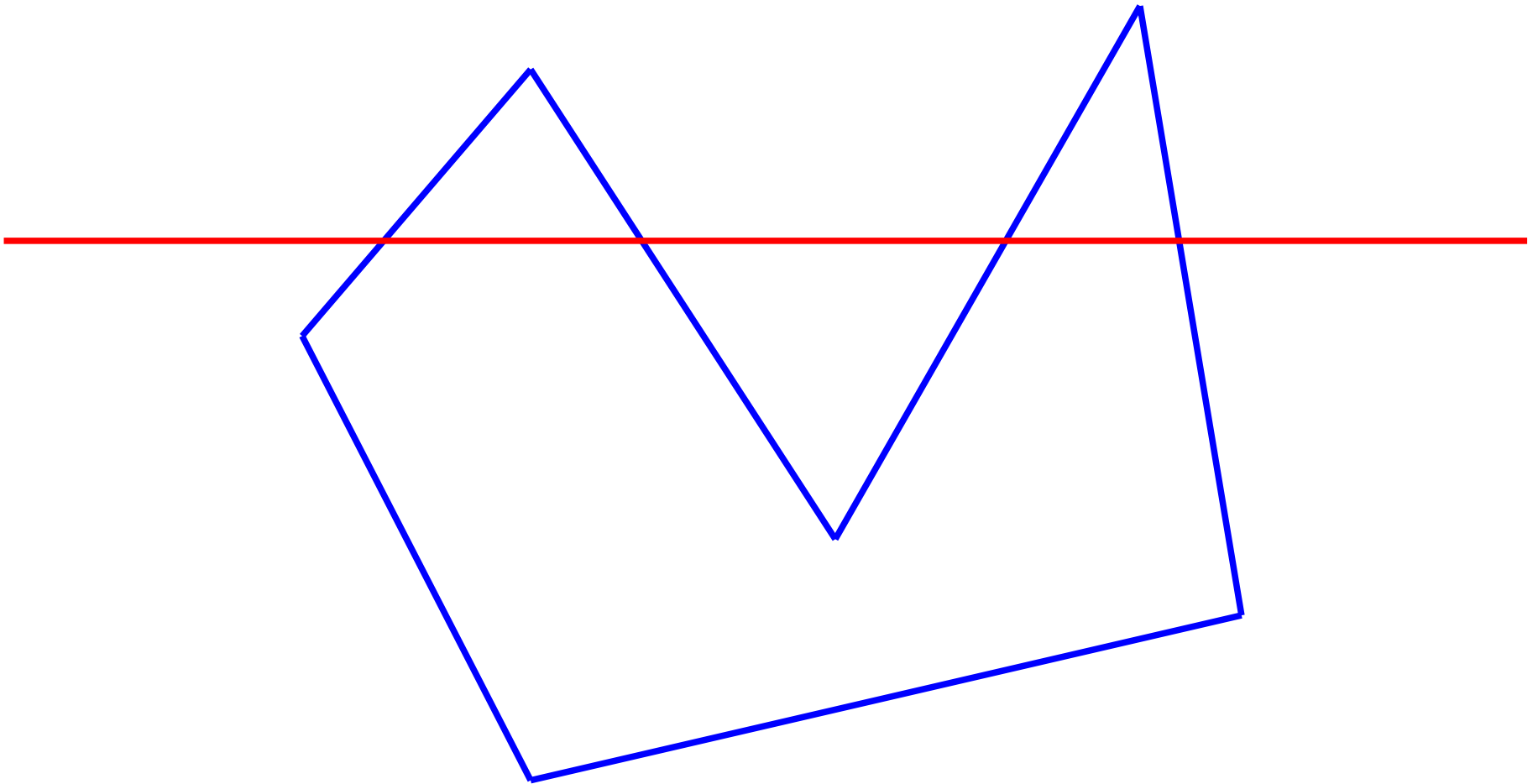


Algoritmo Scan-fill

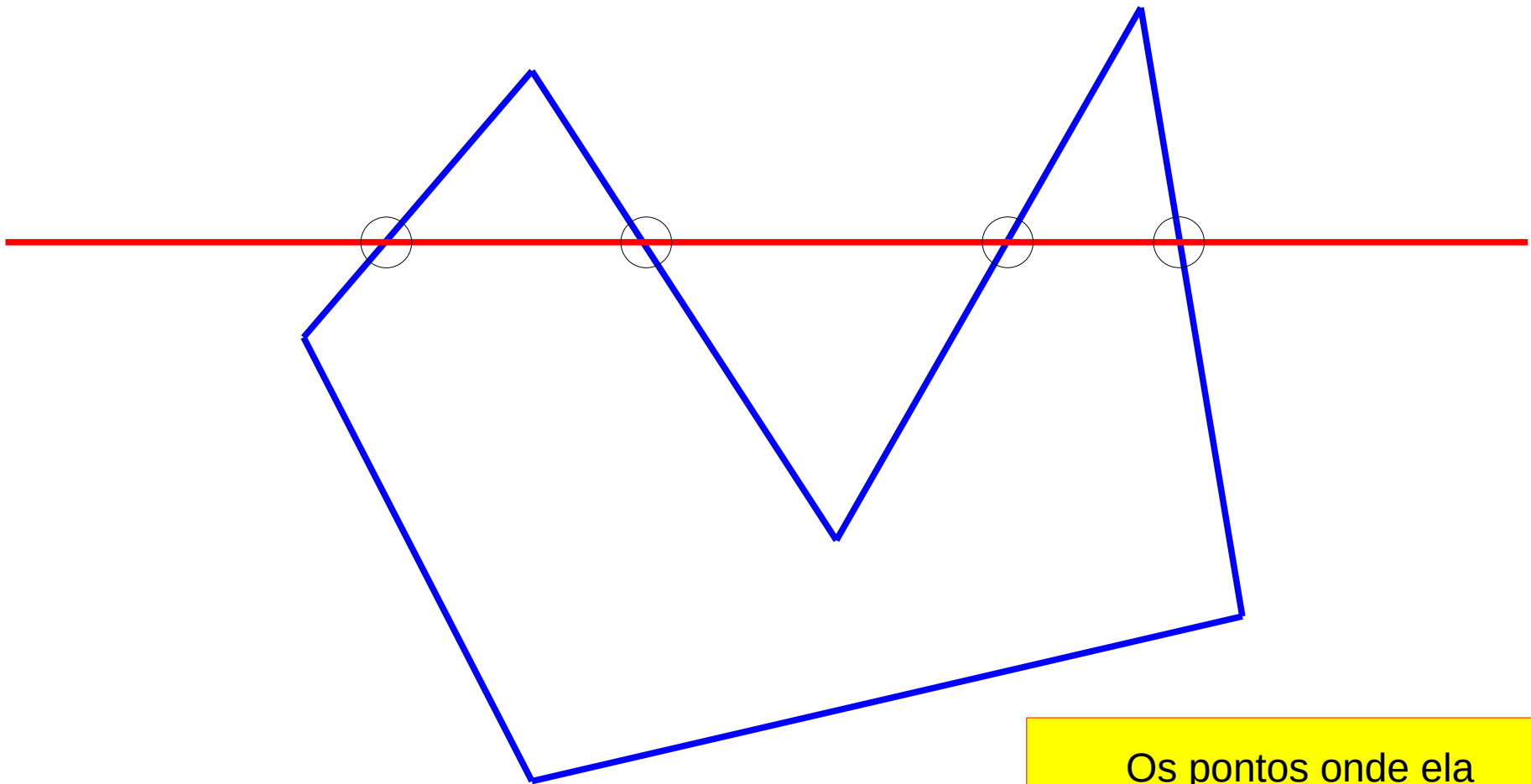


Uma linha horizontal
“varre” o polígono de cima
para baixo

Algoritmo Scan-fill

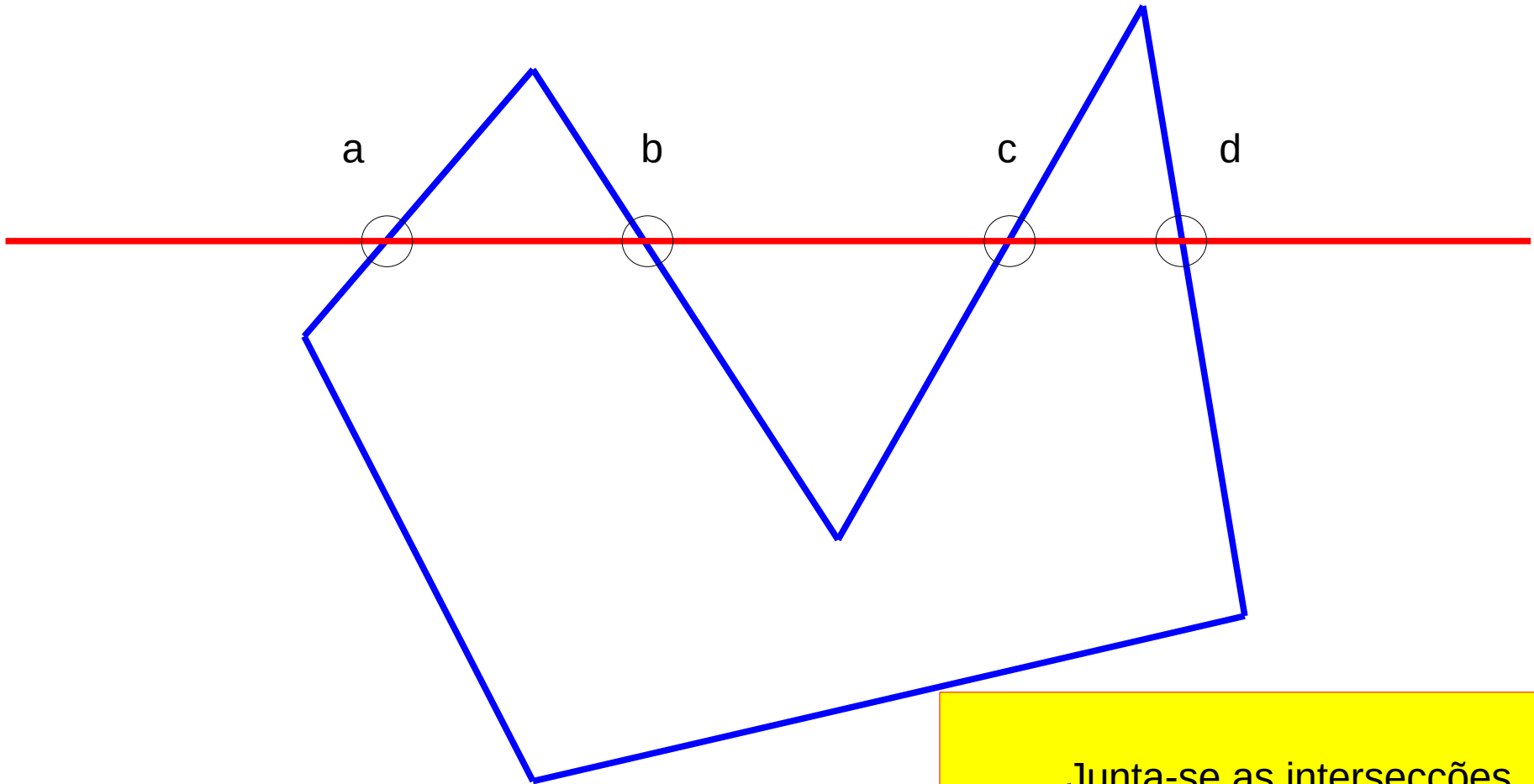


Algoritmo Scan-fill



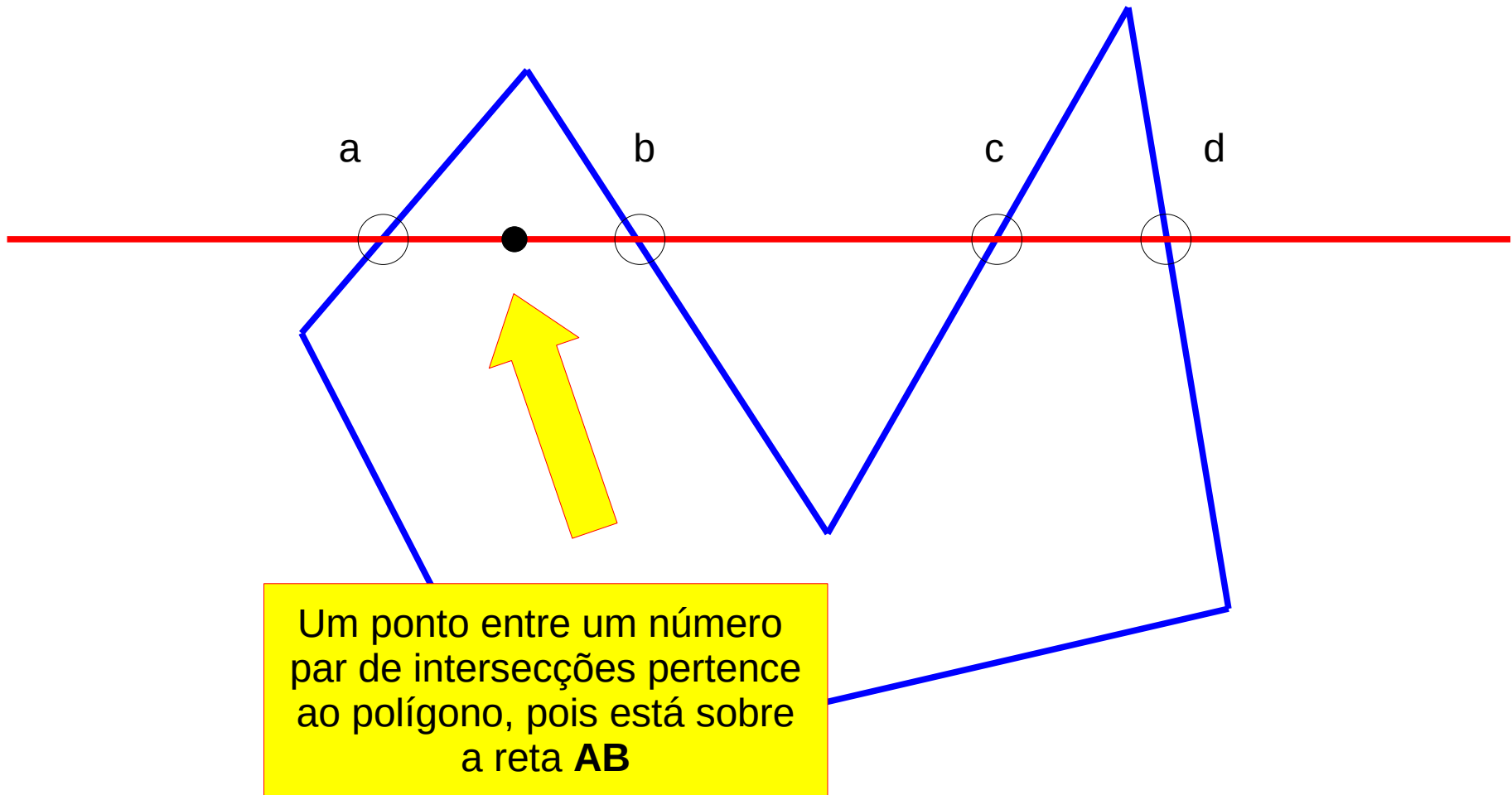
Os pontos onde ela intercepta as linhas do polígono são marcados

Algoritmo Scan-fill

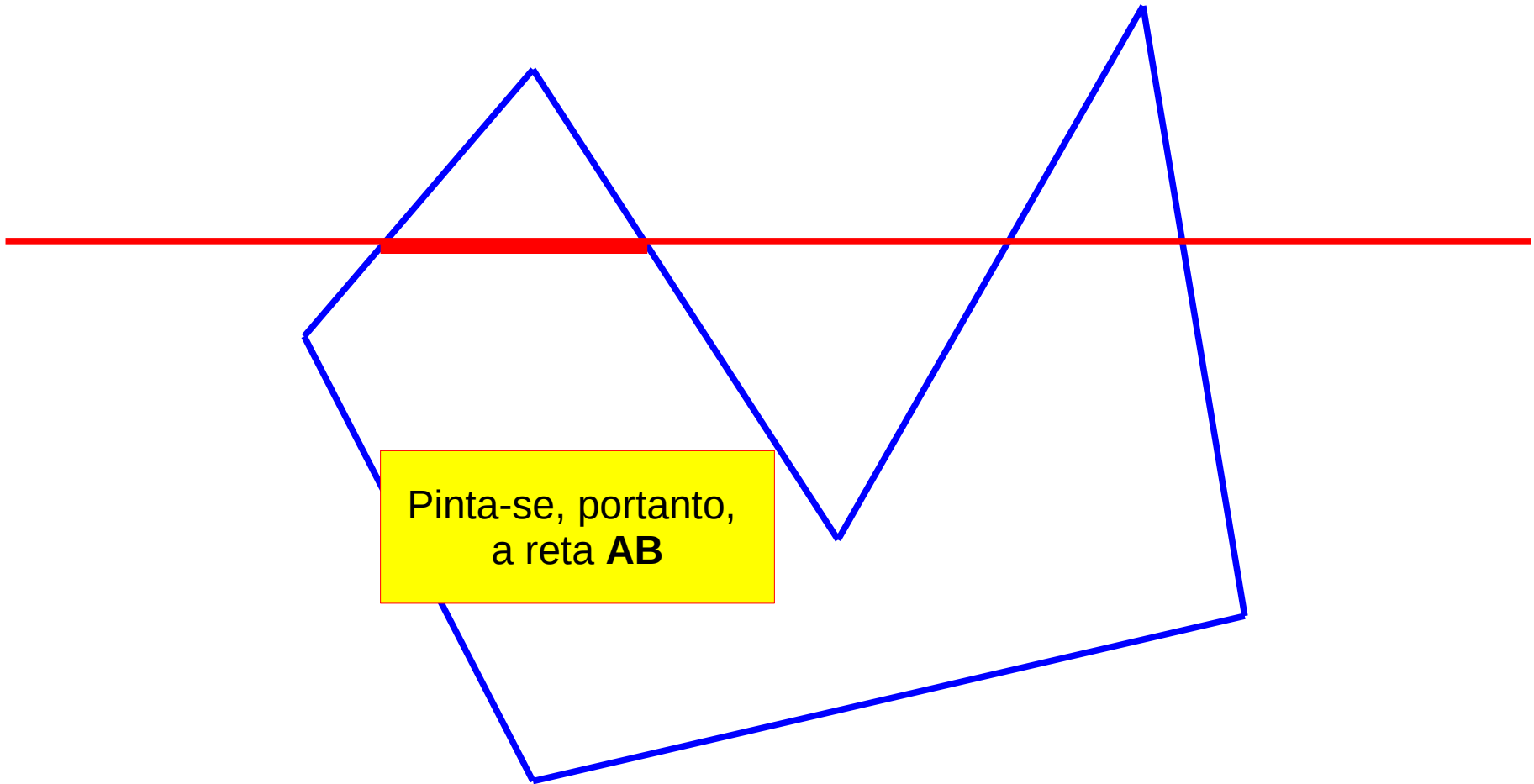


Junta-se as intersecções
aos pares, ordenadas em ordem
crescente de suas coordenadas **x**
Com isso, temos as retas **AB** e **CD**

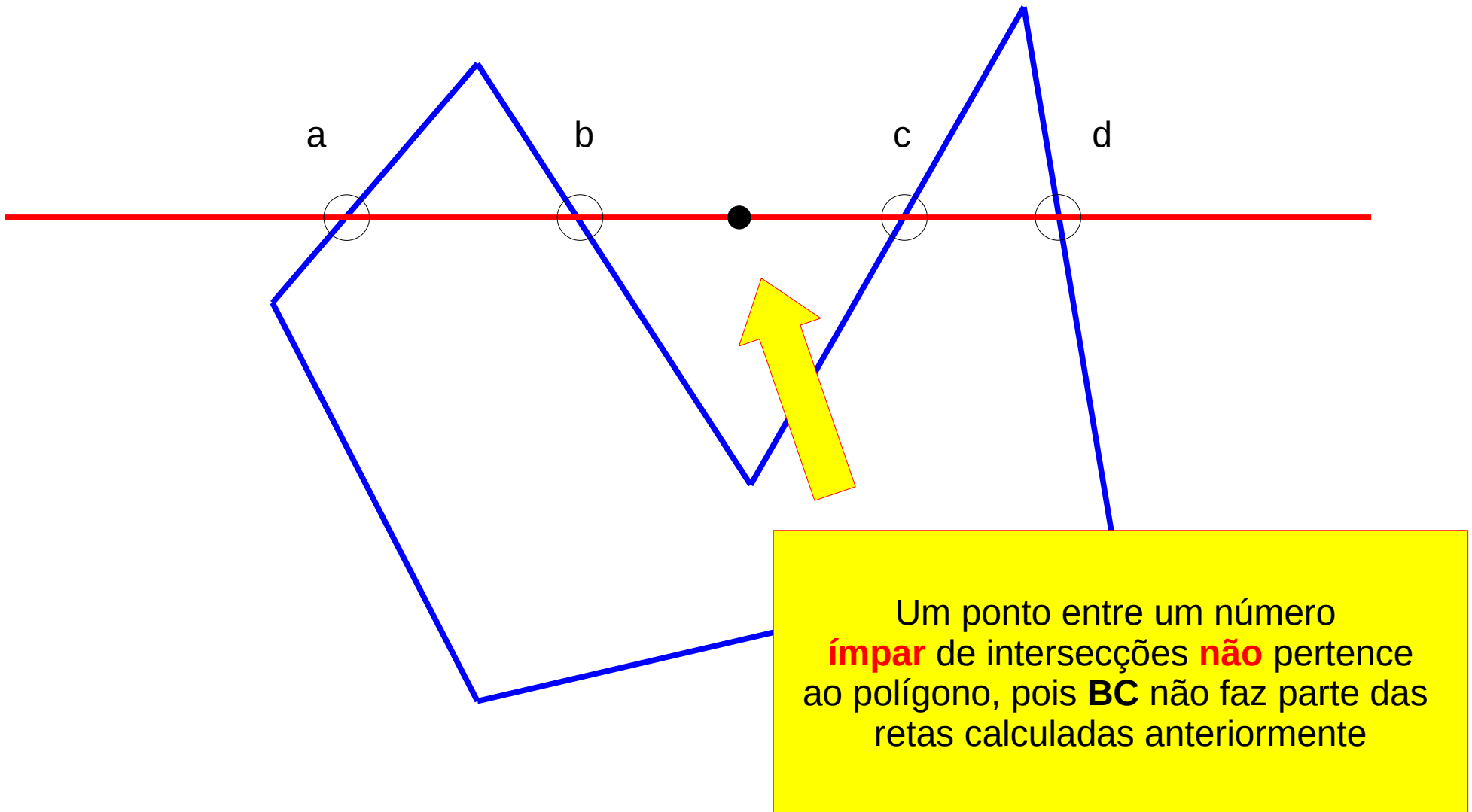
Algoritmo Scan-fill



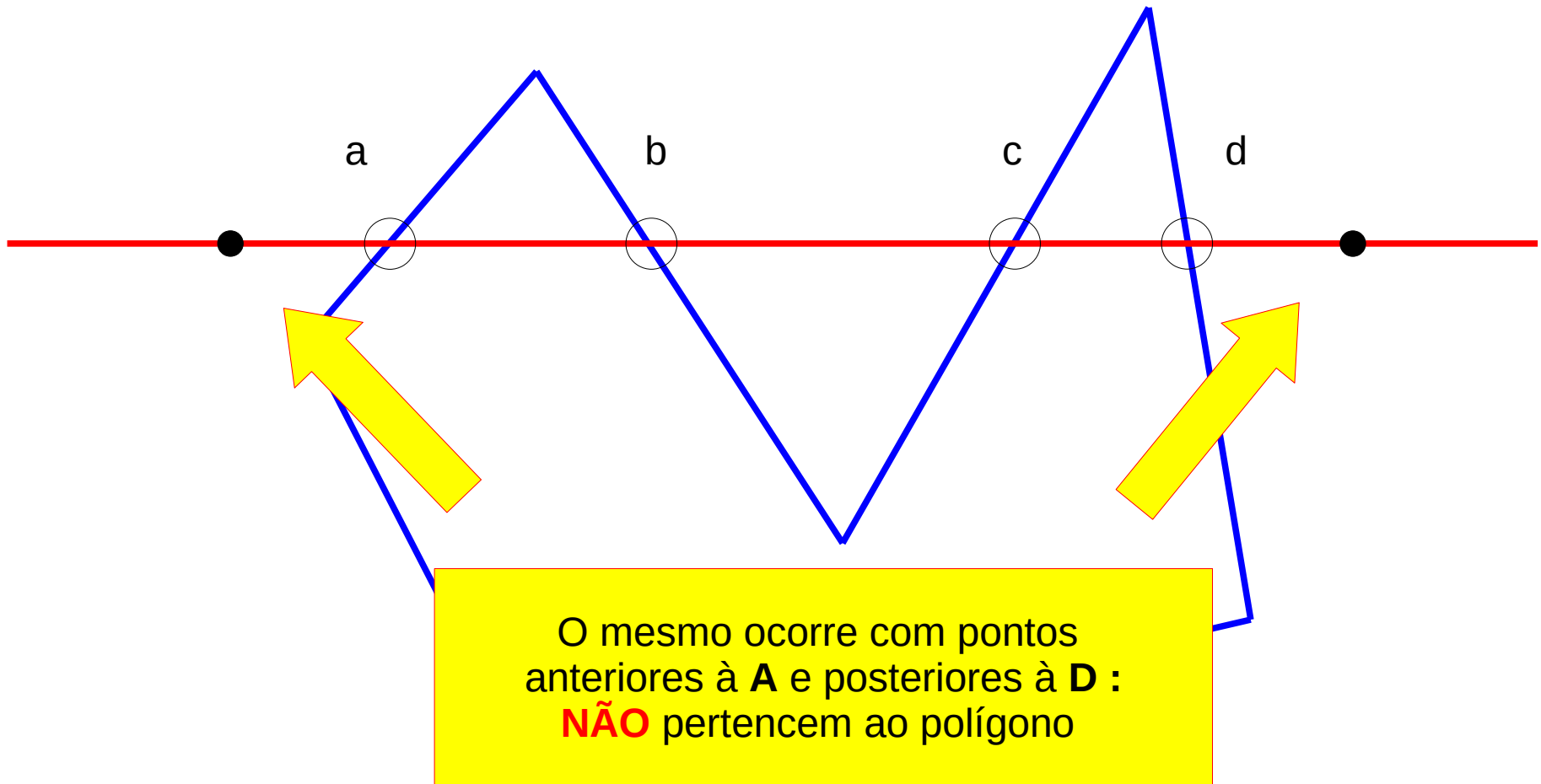
Algoritmo Scan-fill



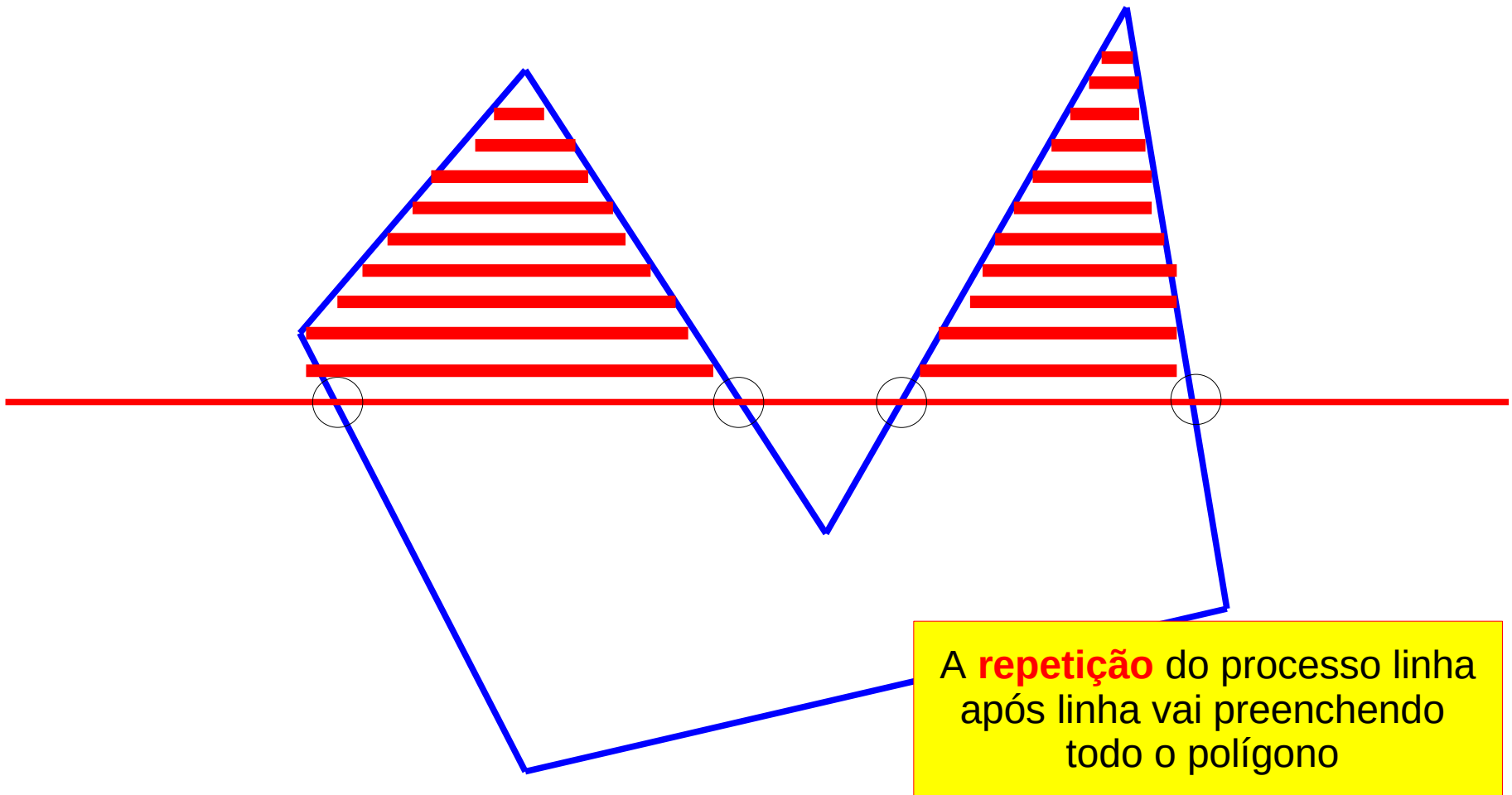
Algoritmo Scan-fill



Algoritmo Scan-fill



Algoritmo Scan-fill



Algoritmo Scan-fill

- Para cada linha da janela faça:
 - Encontre a **interseção** da linha com as bordas do polígono
 - **Ordene as interseções** (pares) pela coordenada x
 - **Trace uma linha entre os pares de interseções consecutivas que estão no interior do polígono,** usando a regra dos pares para decidir se um ponto está no polígono ou não (Um ponto esté dentro do polígono se estiver entre um par de interseções)

Exemplo Scan-fill

- A função ***scanfill*** preenche os pixels contidos dentro do polígono de 4 lados determinado por 4 pares x e y .
- Ele usa uma função de detecção de bordas que simula desenhar a bordas do polígono e grava cada coordenada máxima x para cada coordenada
- Depois para cada coordenada y (a linha) ele desenha uma linha reta até as extremidades gravadas.

Scan-fill (1/2)

```
void scanfill(float x1,float y1,float x2,float y2,float x3,float y3,float x4,float y4)
{
    int le[500],re[500],i,j;

    for(i=0;i<500;i++)
        le[i]=500,re[i]=0;

    edgedetect(x1,y1,x2,y2,le,re);
    edgedetect(x2,y2,x3,y3,le,re);
    edgedetect(x3,y3,x4,y4,le,re);
    edgedetect(x4,y4,x1,y1,le,re);

    for(j=0;j<500;j++)
    {
        if(le[j]<=re[j])
            for(i=le[j];i<re[j];i++)
                drawPixel(i,j, 0);
    }
}
```

Essa implementação é simplificada, e só suporta polígonos com 4 vértices e com até 500 x 500 pixels.

Uma implementação real requer o tamanho total da resolução da tela e alocação dinâmica dos vetores de intersecções

Scan-fill (2/2)

```
void edgedetect(float x1, float y1, float x2, float y2, int *le, int *re)
{
    float temp, x, mx;
    int i;

    if(y1 > y2)
    {
        temp = x1, x1 = x2, x2 = temp;
        temp = y1, y1 = y2, y2 = temp;
    }

    if(y1 == y2)
        mx = x2 - x1;
    else
        mx = (x2 - x1) / (y2 - y1);

    x = x1;

    for(i = int(y1); i <= (int)y2; i++)
    {
        if(x < (float)le[i]) le[i] = (int)x;
        if(x > (float)re[i]) re[i] = (int)x;
        x += mx;
    }
}
```

Scan-fill

- O algoritmo Scan-fill é comumente utilizado em ferramentas do tipo “bucket” (balde)
- Photoshop, Paint, InkScape , Gimp e outras utilizam esse algoritmo



Roteiro

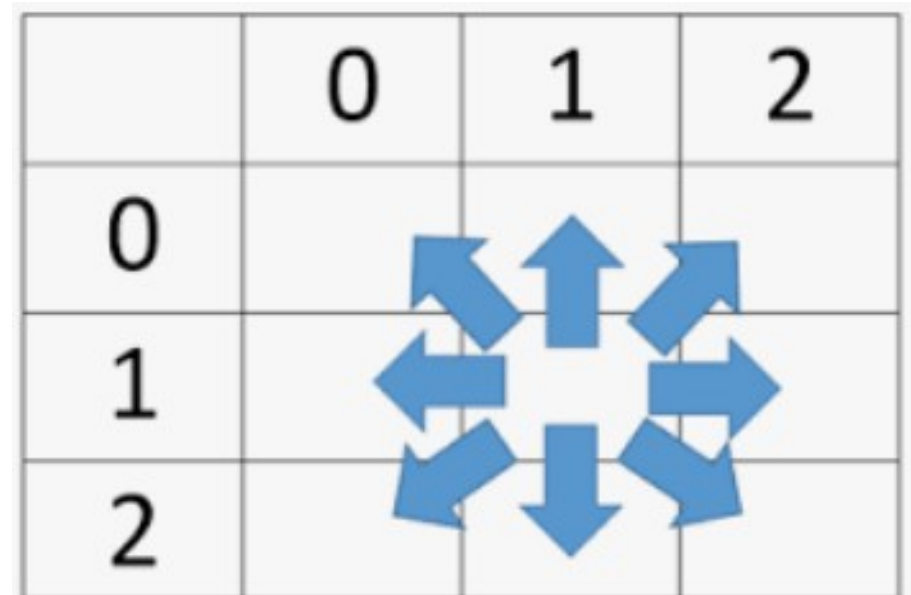
- Objetos paramétricos
 - Círculos
 - Curvas de Bézier
- Preenchimento com cores
 - Scan-fill
 - Flood-fill

Flood-fill

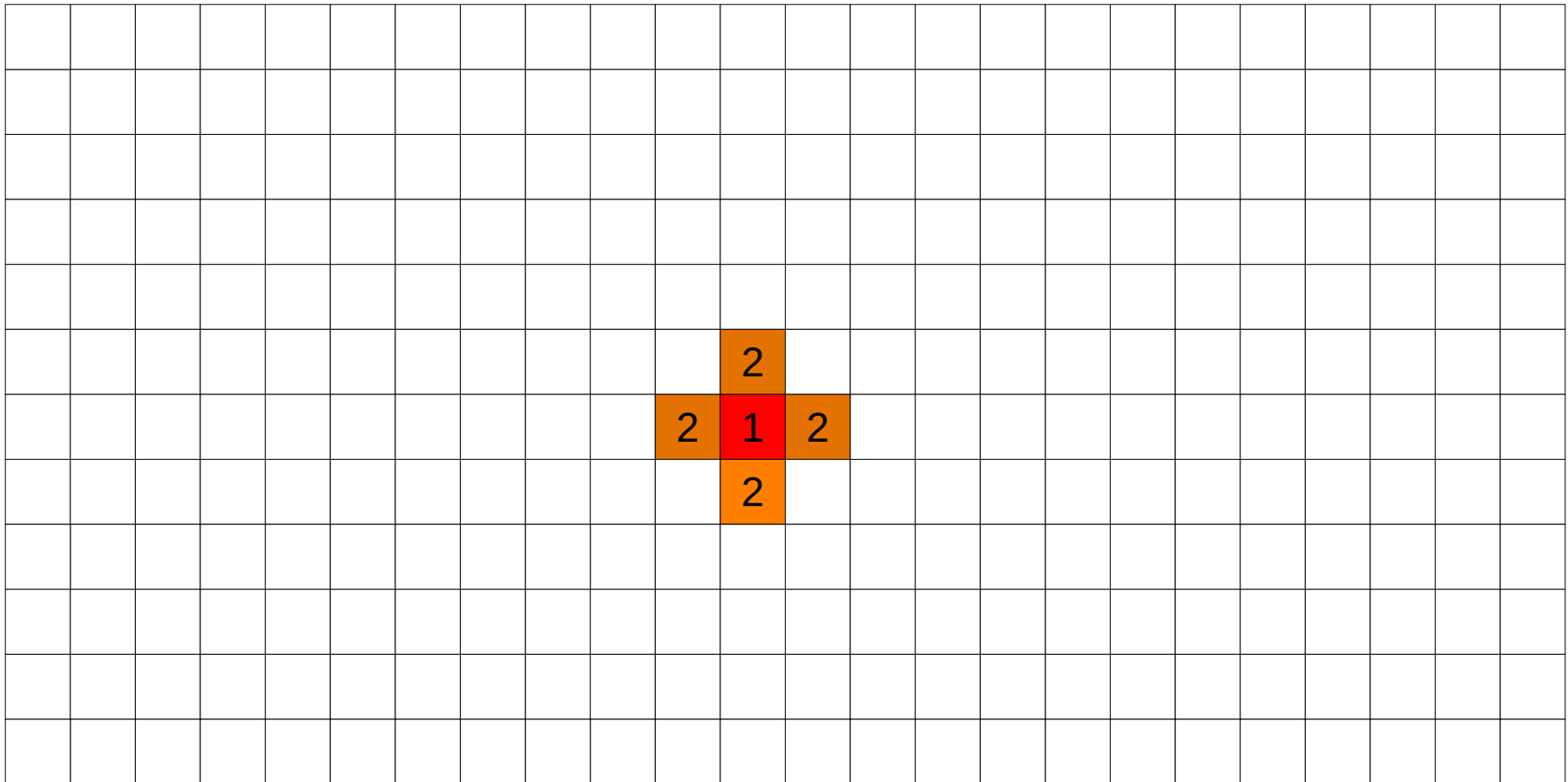
- O algoritmo de ***flood-fill*** é mais simples, porém **sua performance não é tão boa**
- Ele usa uma pesquisa ***recursiva*** para descobrir quais pixels devem ser preenchidos

Floof-fill

- Partindo-se de um ponto, preenche-se os 4 ou 8 pontos que o cercam, e que tenham cores diferentes das bordas



[illegible]



[illegible]

[illegible]

Flood-fill

- A implementação recursiva do algoritmo é demonstrada a seguir **(4 pontos)**:

```
// Implementação com 4 pontos ao entorno
void floodFill(int x,int y,Uint32 newColor, Uint32 oldColor)
{
    if (y< 0 || y > height - 1 || x < 0 || x > width - 1)
        return;

    if(getPixel(x,y)==oldColor)
    {
        setPixel(x,y,newColor);
        floodFill(x+1,y,newColor,oldColor);
        floodFill(x-1,y,newColor,oldColor);
        floodFill(x,y+1,newColor,oldColor);
        floodFill(x,y-1,newColor,oldColor);
    }
}
```

Flood-fill

- A implementação recursiva do algoritmo é demonstrada a seguir **(8 pontos)**:

```
// Implementação com 8 pontos ao entorno
void floodFill(int x,int y,Uint32 newColor, Uint32 oldColor)
{
    if (y< 0 || y > height - 1 || x < 0 || x > width - 1)
        return;

    if(getPixel(x,y)==oldColor)
    {
        setPixel(x,y,newColor);
        floodFill(x+1,y,newColor,oldColor);
        floodFill(x-1,y,newColor,oldColor);
        floodFill(x,y+1,newColor,oldColor);
        floodFill(x,y-1,newColor,oldColor);

        floodFill(x-1,y-1,newColor,oldColor);
        floodFill(x-1,y+1,newColor,oldColor);
        floodFill(x+1,y-1,newColor,oldColor);
        floodFill(x+1,y+1,newColor,oldColor);
    }
}
```

Flood-fill e a pilha

- A implementação recursiva de flood-fill utiliza a pilha do sistema (stack) implicitamente.
- É muito comum gerar erros de **stack overflow**
- A abordagem mais segura é utilizar um objeto pilha, da biblioteca **stack**, para fazer o mesmo.
- Importa-se stack: **#include <stack>**

Flood-fill com *Stack*

- É necessário criar uma estrutura para conter a informação de um ponto

```
typedef struct {int x, y;} Point;
```

- E uma função para transformar pares de coordenadas x e y em pontos:

```
Point getPoint(int x, int y)
{
    Point p;
    p.x = x;
    p.y = y;
    return p;
}
```


Flood-fill com *Stack*

```
void floodFill(int x,int y,Uint32 newColor, Uint32 oldColor)
{
    if (y< 0 || y > height - 1 || x < 0 || x > width - 1)
        return;

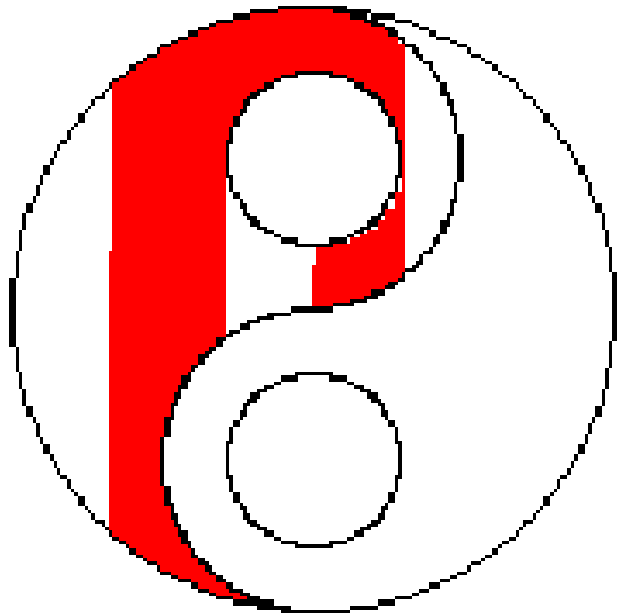
    stack<Point> st;
    st.push(getPoint(x,y));
    while (st.size() > 0)
    {
        Point p = st.top();
        st.pop();
        int x = p.x;
        int y = p.y;
        if (y < 0 || y > height - 1 || x < 0 || x > width - 1)
            continue;

        if (getPixel(x, y) == oldColor)
        {
            setPixel(x, y, newColor);
            st.push(getPoint(x+1,y));
            st.push(getPoint(x-1,y));
            st.push(getPoint(x,y+1));
            st.push(getPoint(x,y-1));
        }
    }
}
```

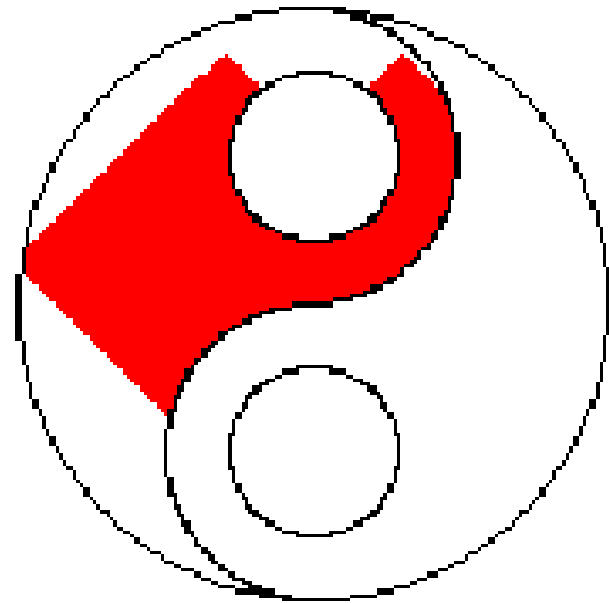
Flood-fill

- O flood-fill pode ser implementado com pilha (**stack**) ou com fila (**queue**)
- O interessante é que a maneira como a figura é preenchida varia com cada um das estruturas de dados, mas a performance é a mesma

Flood-fill



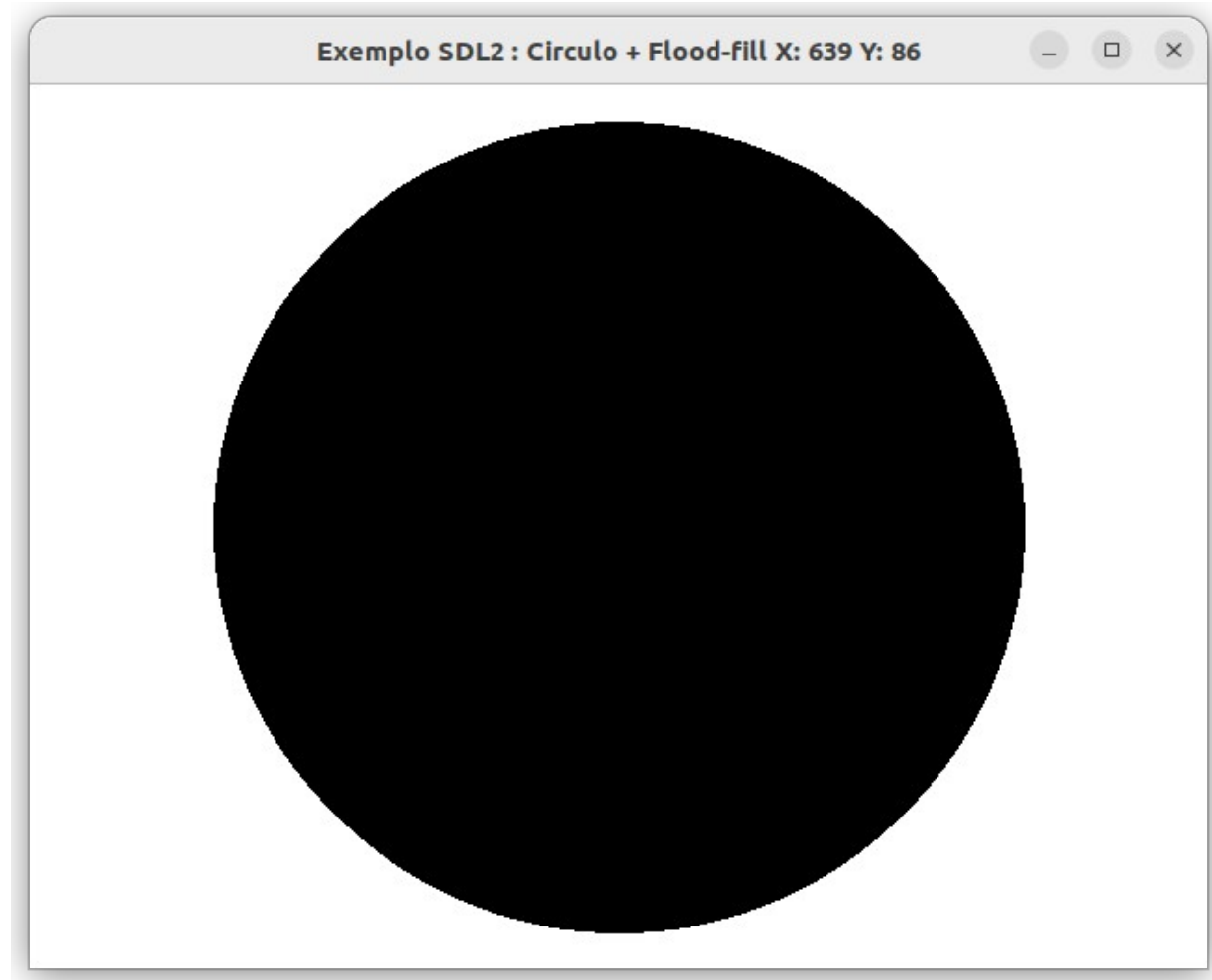
Com pilha



Com fila

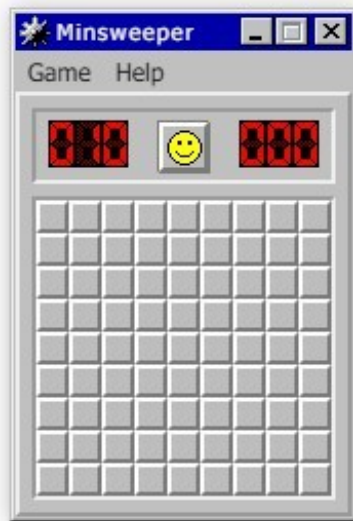
Flood-fill : Exemplo

```
void display()  
{  
    Uint32 cor = RGB(0,0,0); // Preto  
    drawBresenhamCircle(320,240,220);  
    floodFill(320,240,cor);  
}
```



Finalizando

- A performance do Scan-fill é superior a do Flood-fill porque no primeiro cada pixel é testado **apenas uma vez**
- Softwares e placas gráficas implementam **Scan-fill**
- **Flood-fill** é utilizado em jogos como Campo minado (Minesweeper) e Candy Crush



Dúvidas?



Atividade 1 :

- Crie um projeto chamado **SDL_Circle**, e implemente nele o algoritmo de **desenho de círculos de Bresenham**
 - (Reaproveite o código do projeto **SDL_Points** fornecido para isso)
- Desenhe ao menos cinco círculos, em coordenadas diferentes da janela

Atividade 2 :

- Crie um projeto chamado **SDL_Bezier**, e implemente nele o algoritmo de desenho curvas de Bézier.
- O programa deverá solicitar ao usuário que informe **4 pares de coordenadas (x,y)** e **desenhe uma curva de Bézier com esses valores**

Atividade 3 :

- Crie um projeto chamado **SDL_Flood**, e implemente nele o algoritmo de **Flood-fill** utilizando **pilhas**.
- Divida sua janela em quatro partes usando linhas retas e preencha cada parte com uma cor diferente



Atividade 4

- Crie um novo projeto, chamado **SDL_Heart**, e, combinando todas as funções dos três exercícios anteriores, faça um programa que implemente a figura a seguir:

